

Desarrollo de una celda reconfigurable de CGRA

Duarte Sanchez David, Jimenez Ulate Eddy, Ruiz Rivera Jeffrey, Esquivel Arias Fabian Zúñiga Ramírez Jose Eduardo
davidduarte28, e.jimenez.10, yr1ruiz, fabiesqui22, je.zuniga@estudiantec.cr

I. INTRODUCCIÓN

Las arquitecturas reconfigurables han sido muy exitosas en la industria debido a que la evolución del software ha presentado una rápida evolución, con diversas aplicaciones, distintas necesidades de usuario y progreso científico, por lo que el hardware no se puede adaptar en la misma velocidad al software [1]. Sistemas de propósito específico pueden sufrir de obsolescencia rápida y si el precio del desarrollo y producción del hardware es alto, se vuelve inviable económicamente [1].

Actualmente, arquitecturas como las FPGA (*Field Programmable Gate Arrays*) han sido ampliamente adoptadas en diversas aplicaciones debido a su flexibilidad y capacidad de personalización, tomando fuerza en Computación de alto desempeño (HPC) por sus siglas en inglés. Sin embargo, las FPGAs presentan ciertas limitaciones como tiempos de configuración lentos, largos tiempos de compilación y aunque es algo relativo, bajas frecuencias de operación [2].

Debido a lo anterior, se ha tomado un interés en HPC en utilizar arquitecturas como los CGRAs (*Coarse-Grained Reconfigurable Arrays*) que ofrecen una solución intermedia entre los FPGAs y los procesadores de propósito general (GPP). Los CGRAs son arquitecturas de hardware que conceptualmente permiten que el silicio sea maleable y su funcionalidad pueda reconfigurarse de manera dinámica [2]. Los CGRA tienen la ventaja de ser uno o dos ordenes de magnitud más eficientes que las FPGAs y más de 2 a 3 ordenes de magnitud más eficientes que los GPPs [1].

El desarrollo de una celda reconfigurable de CGRA es de gran importancia por su relevancia no solo en HPC, si no también en su adopción en aplicaciones de *Machine-learning*, procesamiento de imágenes y visión por computadora, comunicaciones y otras áreas [3]. Por lo tanto el objetivo de este trabajo es el desarrollo de un CGRA heterogéneo, que posea tentativamente celdas de ruteo de datos, celdas con bancos de memoria distribuida, celdas de procesamiento escalar y celdas de procesamiento vectorial, con la capacidad de ejecutar programas utilizando el set de instrucciones RISC-V, con la capacidad de ejecutar lógica de ocho, 16 y 32 bits.

La importancia de utilizar este set de instrucciones es que este es un estándar abierto, facilitando su adopción debido a que no requiere pagos por licencias. Este es un set de instrucciones basado en la simplicidad, haciendo que el diseño de hardware sea sencillo y una implementación más directa [4].

II. ANTECEDENTES Y TRABAJO RELACIONADO

En los últimos años, las CGRA han ganado un protagonismo significativo como aceleradores de hardware debido a su

capacidad para ofrecer un equilibrio excepcional entre la flexibilidad del software y el alto rendimiento del hardware de aplicación específica [1], [2]. A diferencia de las arquitecturas de grano fino, las CGRAs operan a nivel de palabra o bloque, lo que reduce sustancialmente la sobrecarga de reconfiguración y enrutamiento, haciéndolas particularmente atractivas para aplicaciones eficientes en el borde (edge computing). El desarrollo e investigación de estas arquitecturas ha sido fuertemente impulsado por la adopción del repertorio de instrucciones (ISA) de código abierto RISC-V [4], cuya naturaleza extensible facilita la creación de Sistemas en Chip (SoC) heterogéneos altamente personalizados.

Investigaciones recientes han demostrado las ventajas de integrar estrechamente núcleos RISC-V con arreglos CGRA para superar el cuello de botella tradicional de Von Neumann. Por ejemplo, [5] exploran la integración de un acelerador CGRA acoplado a un núcleo de alto rendimiento RISC-V CVA6 enfocado en ecosistemas cloud-edge, mientras que [3] proponen DVHetero, un entorno de trabajo integral para el diseño y la validación de SoCs heterogéneos basados en esta combinación.

A nivel de implementación práctica, estas arquitecturas se materializan en esfuerzos de código abierto de vanguardia; repositorios como risc-v-cgra (desarrollado por ECASLab) investigan estas integraciones a nivel de sistema, mientras que proyectos como OpenEdgeCGRA (ESL-EPFL) proporcionan plataformas open-hardware completas. En este último, la arquitectura se compone de una malla bidimensional configurable de Elementos de Procesamiento (PEs) o celdas reconfigurables que interactúan en forma de toroide y comparten registros para maximizar la paralelización de instrucciones con una sobrecarga de memoria mínima. Otro proyecto destacable como OpenCGRA desarrollado por el PNNL (*Pacific Northwest National Laboratory*) el cual genera procesadores reconfigurables de forma parametrizable, de esta manera, permite diseñar el hardware por código como el tamaño de la malla, el tipo de celdas, cómo se comunican entre ellas, luego se prueba el funcionamiento usando Python y Verilator y finalmente se exporta el diseño a un código de diseño de hardware estándar (HDL) como Verilog.

Para que este tipo de sistemas alcance su máximo potencial, el diseño interno de la celda reconfigurable individual (PE) y las herramientas de compilación asociadas son críticos [6]. Destacan la necesidad de un mapeo consciente del contexto de memoria (context-memory aware mapping) para asegurar una aceleración eficiente desde el punto de vista energético, subrayando que la latencia en el acceso a datos dicta el

rendimiento global. En este marco, el diseño y desarrollo de una nueva celda reconfigurable de CGRA se fundamenta en la necesidad de optimizar las interconexiones locales, el acceso determinista a operandos intermedios y la versatilidad de sus operaciones a nivel de ciclo de reloj, retos que continúan siendo áreas de activa exploración en la frontera del diseño de aceleradores heterogéneos.

III. ANÁLISIS DE SOLUCIONES EXISTENTES (SOTA)

En la actualidad, existen distintos tipos de CGRAs, donde estos pueden tener un arreglo de PEs homogéneos como es el caso de los CGRA HyCUBE y los DySER [7] o pueden ser de tipo hererogéneos como el Plasticine que posee dos tipos de unidad, los PCU y los PMU [8]. Otra característica importante a destacar es el tipo de precisión que estos dispositivos pueden soportar, donde muchos de estos están restringidos a solo procesar operaciones de enteros, siendo estos más de propósito general y otros donde si se añade la ventaja de punto flotante para procesos más especializados [9].

El tipo de construcción de las CGRA ha tomado mucha importancia debido a los sistemas HPC *High Performance Computing*, donde estos se ven altamente beneficiados de que los PEs sean de tipo heterogéneo, donde se tenga una mezcla de PEs basicos y complejos [9]. Esto es es altamente importante para funciones matemáticas debido a que las CGRAs tradicionales no lo manejan de manera óptima

IV. PROPUESTA DE SOLUCIÓN Y ARQUITECTURA DE SEGUNDO NIVEL

IV-A. Descripción General y Alcance

Presenta una **celda reconfigurable heterogénea** para CGRA, superando arquitecturas homogéneas con sub-utilización ante cargas heterogéneas [10], [11]. Introduce **cuatro tipos de celdas especializadas** en un único *fabric*:

1. **Enrutamiento:** FIFOs para flujo dinámico sin bloqueos.
2. **Memoria distribuida:** Bancos SRAM locales (1–4 KB) eliminan cuellos de botella.
3. **Procesamiento escalar:** ALU multi-precisión (int8/16/32) + bucles hardware.
4. **Procesamiento vectorial:** SIMD float32 (4–8 lanes) para 4–8 elementos simultáneos.

La Tabla I mapea desafíos del SOTA a soluciones.

Cuadro I
DESAFÍOS SOTA Y SOLUCIONES PROPUESTAS.

ID	Desafío	Solución
C1	PEs homogéneos → sub-utilización	4 celdas especializadas (enrut., mem., esc., vec.)
C2	Overhead routing > 30 % área	Celdas routing dedicadas con FIFOs
C3	Memoria centralizada → cuellos	SRAM distribuido local (1–4 KB)
C4	Latencia config. RISC-V	CSR + DMA; config. parcial por contexto
C5	Multi-precisión simultánea	ALU escalar int8/16/32; VALU float32
C9	NoC → cuello de botella	Malla 2D, enlaces 32 b, FIFOs por crédito

IV-A1. Novedad Arquitectónica: Combinación inédita: (a) routing como celda primaria con FIFOs, (b) memoria distribuida por celda, (c) multi-precisión entero-flotante en celdas diferenciadas, (d) acoplamiento nativo RISC-V (RV32IMAF) via CSR + DMA. Ninguna arquitectura revisada (ADRES,

HyCUBE, Plasticine, etc.) integra simultáneamente estas 4 características.

IV-A2. Alcance y Entregables:

- **RTL (VHDL):** 4 celdas + router NoC básico.
- **Integración:** Arreglo 2×2 con NoC en malla.
- **Verificación:** Testbench exhaustivo + programas aleatorios [12].
- **Síntesis FPGA:** Segun disponibilidad (recursos + $F_{m\acute{a}x}$).
- **Benchmarks:** Multiplicación matrices, FFT, reducción vectores.

Métricas en Tabla II.

Cuadro II
MÉTRICAS DE EVALUACIÓN (FLUJO HLS + FPGA).

Métrica	Unidad	Herramienta
Recursos FPGA	LUT/FF/DSP/BRAM	Vitis HLS / Vivado 2023.1
$F_{m\acute{a}x}$	MHz	Vivado Timing
Latencia	ciclos	C/RTL Co-simulación (HLS)
Throughput	OP/s, GFlop/s	Análisis analítico ($F_{m\acute{a}x}$ + latencia)
Potencia	mW	Vivado Power
Speedup SW	×	SystemC vs. RISC-V (C++)

IV-B. Arquitectura de Segundo Nivel: Componentes

Topología malla 2×2 con NoC (control por créditos). Fig. 1 muestra el diagrama.

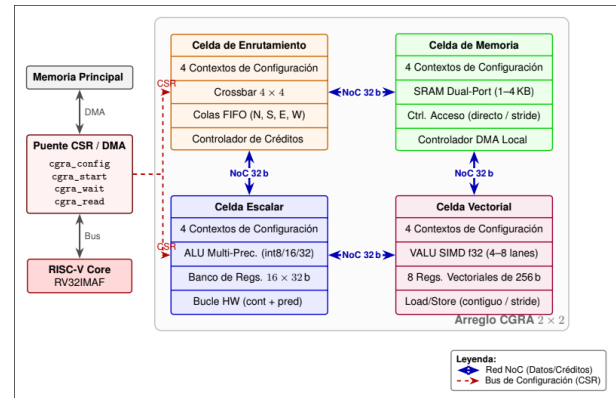


Figura 1. Diagrama de arquitectura de segundo nivel generado con inteligencia artificial.

IV-B1. Celda de Enrutamiento: Primera clase (no multiplexores estáticos): FIFOs 4–8 entradas $\times$ 32 b por puerto; crossbar 4×4 (1 ciclo, multicast); control por créditos; 4 contextos (cambio 1 ciclo). Overhead routing < 15 % área (vs. ~ 30 % en HyCUBE [11]).

IV-B2. Celda de Memoria Distribuida: SRAM dual-port 1–4 KB (BRAM FPGA); controlador (directo/indirecto/stride); interfaz NoC (1–2 ciclos); DMA local (transferencias bloque sin host). Elimina contención centralizada.

IV-B3. Celda Escalar: ALU int8/16/32 (4 ops SIMD int8/ciclo); 16 registros $\times$ 32 b; bucles HW (contador + predicación); 4 contextos; interfaz NoC 32 b por crédito.

IV-B4. Celda Vectorial: VALU SIMD 4–8 lanes float32 (suma, MAC, reduce); 8 reg. vectoriales (128/256 b); carga/alm. strided; 4 contextos; NoC 128 b (modo ráfaga). Consumirá $\sim 3 \times$ DSP, entregará $\sim 4 \times$ throughput float32 [9].

IV-B5. Router NoC: Malla 2D, enlaces 32 b; créditos + FIFO 4 flits; arbitraje round-robin; 1 ciclo/hop; puerto host CSR/DMA (config. celdas, transferencias DMA, lectura resultados).

IV-B6. Interfaz RISC-V: CSR dedicado (config. celda, contexto, start/stop); DMA 1 canal; interrupciones/flags; 4 instrucciones custom (cgra_config/start/wait/read). Reduce latencia config. (desafío C4) [?], [13].

V. PLAN DE EXPERIMENTACIÓN Y METODOLOGÍA

V-A. Test y estímulo

Para probar un diseño electrónico se necesita un testbench, encargado de dirigir un estímulo hacia el diseño y de facilitar el monitoreo de lo que ocurre en el diseño bajo prueba (DUT). Como se ha hecho en otros diseños, y en los distintos frameworks de desarrollo para CGRA, resulta necesario contar con un algoritmo que la arquitectura sea capaz de ejecutar. Para esto se suele partir de los llamados kernels: algoritmos representativos, típicamente codificados en C (por ejemplo, FFT y GEMM), anotados con directivas de compilación sobre los loops, cuya estructura de flujo de datos puede mapearse sobre el arreglo de PEs de una CGRA. Este es, de hecho, el enfoque que sigue Morpher dentro del ecosistema OpenCGRA: a partir del kernel en C genera el grafo de flujo de datos (DFG) y el layout de memoria scratchpad, y produce los vectores de prueba que alimentan la etapa de simulación/emulación, con los que se verifica la correctitud funcional y se obtienen estimaciones de área y potencia [14]. El uso de FFT como kernel de referencia es además consistente con la literatura de CGRA orientada a procesamiento de señales, donde aparece de forma recurrente junto con rutinas como IDCT, FIR e IIR [2].

Vale la pena notar que incluso en frameworks consolidados como OpenCGRA, sus propios autores señalan que el simulador depende de testbenches escritos manualmente por el usuario y no automatiza la generación de datos de prueba ni las verificaciones de extremo a extremo [14]. Esto refuerza la pertinencia de plantear, desde un inicio, un flujo de pruebas estructurado y progresivo como el que se propone a continuación.

Si bien un objetivo valioso es lograr ejecutar un algoritmo como FFT o GEMM compilado de principio a fin —desde el programa en C hasta el bitstream de configuración y los datos de entrada de la CGRA—, dado el tiempo disponible para el desarrollo del proyecto, el primer objetivo apunta a pruebas algorítmicamente más sencillas. Se proponen los siguientes pasos:

1. **Una sola PE ejecutando $c = a + b$ (constante) o $c = a \times b$.** Esto valida la carga de la palabra de configuración (config word), la lectura de registros/entradas, la operación de la ALU y la escritura del resultado de salida. Es, literalmente, el “hello world” de la arquitectura.
2. **Cadenas de operaciones, en modo pipeline:** suma acumulada ($y = x_0 + x_1 + x_2 + \dots + x_n$) y potencias por cuadrados sucesivos ($x, x^2, x^4, x^8, \dots$).
3. **MAC simple ($acc += a \times b$):** es el bloque base de casi todo kernel real (FFT, GEMM, convoluciones), por

lo que si esta prueba funciona correctamente, se gana confianza para escalar hacia diseños más complejos.

Una vez superadas estas pruebas básicas, se puede avanzar hacia benchmarks típicos de la literatura de CGRA, como MachSuite y PolyBench [15], [16], que junto con Wavelib y BEEBS conforman el conjunto de suites utilizado por Morpher para evaluar su flujo de compilación [14]. Estas suites proveen bibliotecas de kernels (incluyendo FFT y GEMM) que pueden mapearse sobre la CGRA mediante un flujo de compilación similar al descrito anteriormente.

V-B. Monitoreo

Dado que este proyecto se desarrollará en SystemC, se busca aprovechar las funcionalidades propias del lenguaje para implementar la medición de desempeño. Se propone implementar un monitor como un componente `SC_MODULE` que observe, de manera no intrusiva, elementos del DUT como señales y contadores internos. Esta separación entre la lógica funcional y la lógica de medición sigue el principio de diseño modular propio de SystemC, en el que el monitor puede añadirse o removerse del testbench sin modificar el módulo bajo análisis.

Se propone que el monitor calcule, como mínimo, las siguientes métricas:

- **Operaciones por segundo (OP/s):** cociente entre el número total de operaciones ejecutadas por el DUT y el tiempo total de ejecución.
- **Latencia promedio:** registrando `sc_time_stamp()` al inicio y al final de cada operación individual, promediando sobre la ventana de muestreo.
- **Tasa de utilización:** proporción de ciclos activos respecto al total de ciclos.

Estas métricas se imprimen a consola mediante `SC_REPORT_INFO`.

Además, cuando el diseño alcance una etapa apta para síntesis en FPGA o ASIC, será posible medir métricas de potencia, área y temporización (*timing*) con las herramientas EDA correspondientes. Esto aplica tanto para la AMD Kria KV260 como para la AMD Alveo U55C.

V-C. Experimentación arquitectónica

Ligado a la exploración del espacio de diseño, se plantea generar variaciones del diseño arquitectónico de la CGRA, modificando la cantidad y el tipo de PEs (*processing elements*). Por ejemplo, cambios en la distribución de tipos de PE (más bancos de memoria, más ALUs de punto fijo, más ALUs de punto flotante), así como distintas dimensiones $n \times n$ del arreglo, orientadas a optimizar la ejecución de instrucciones vectoriales.

Este tipo de exploración es consistente con la diversidad de arquitecturas CGRA reportada en la literatura, donde el desempeño depende fuertemente de la composición y disposición de los PEs [2].

Cuadro III
RESUMEN DE COMPONENTES.

Componente	Función	Parámetros clave
Enrutamiento	Interconexión + buffering	FIFOs 4–8 × 32 b; crossbar 4 × 4; 4 contexto
Memoria	SRAM distribuido	1–4 KB dual-port; DMA; 1–2 ciclos
Escalar	Entero multi-precisión	int8/16/32; 16 reg.; bucles HW; 4 contexto
Vectorial	SIMD float32	4–8 lanes; 8 reg.; MAC; 4 contexto
Router NoC	Transporte malla	32 b; créditos; 1 ciclo/hop; round-robin
RISC-V	Host-acelerador	CSR + DMA + ISA custom; RV32IMAF

V-D. Herramientas y Entorno de Desarrollo

Con el fin de asegurar la reproducibilidad estricta de las pruebas de diseño y simulación, se define la infraestructura de herramientas de software con sus respectivas versiones en la Tabla IV.

Cuadro IV
HERRAMIENTAS DE DESARROLLO Y VERSIONES DEL ENTORNO

Herramienta / Software	Versión	Propósito en el Experimento
SystemC	3.0.1	Modelado y simulación de la arquitectura CGRA.
Xilinx Vivado Design Suite	2025.2	Síntesis lógica y análisis de timing.
ModelSim / Questa Advanced	2025.1	Simulación funcional RTL ciclo a ciclo.

DECLARACIÓN DE USO DE HERRAMIENTAS DE IA

Para este trabajo se hizo uso de Inteligencia Artificial para la ayuda en la estructura de la introducción y el estado del arte, así como para una revisión gramatical y ortográfica del documento.

REFERENCIAS

- [1] L. Liu, J. Zhu, Z. Li, Y. Lu, Y. Deng, J. H. Han, S. Yin, and S. Wei, “A survey of coarse-grained reconfigurable architecture and design: Taxonomy, challenges, and applications,” *ACM Computing Surveys*, vol. 52, pp. 1 – 39, 2019.
- [2] K. S. Artur Podobas and S. Matsuoka, “A survey on coarse-grained reconfigurable architectures from a performance perspective,” *IEEE ACCESS*, vol. 8, 2020.
- [3] G. Zhu, L. Deng, K. Zhang, W. Fan, B. Jin, W. Cao, F. Zhang, X. Zhou, F. Zhang, and X. Yu, “Dvhetero: A framework for designing and validating heterogeneous soc with risc-v processor and cgra,” *ACM Transactions on Reconfigurable Technology and Systems*, vol. 18, pp. 1 – 30, 2025. [Online]. Available: <https://www.semanticscholar.org/paper/8d8821ac4c9ec42fc45e662ff65a0ccd6361297a>
- [4] EdgeIR Staff. (2024, jan) What is risc-v and why is it important? online. Accessed: 2026-06-11. [Online]. Available: <https://www.edgeir.com/what-is-risc-v-and-why-is-it-important-20240109>
- [5] J. Granja, D. Vázquez, A. Rodríguez, and A. Otero, “Integration of a cgra accelerator with a cva6 risc-v core for the cloud-edge,” null. [Online]. Available: <https://www.semanticscholar.org/paper/b4dfe9170dd1bfa04c4eee19794ebe685b388fe0>
- [6] S. Das, K. J. M. Martin, and P. Coussy, “Context-memory aware mapping for energy efficient acceleration with cgras,” *Design, Automation & Test in Europe Conference & Exhibition*, vol. null, pp. 336–341, 2019. [Online]. Available: <https://www.semanticscholar.org/paper/b3c002d29c59700cc1827aff29a72b560c86346a>
- [7] M. Wijnvliet, H. Corporaal, and A. Kumar, *Blocks, Towards Energy-efficient, Coarse-grained Reconfigurable Architectures*. Springer Nature Switzerland AG, 2022.
- [8] , “Plasticine: A reconfigurable architecture for parallel patterns,” in , 2017, stanford.

- [9] E. Del Sozzo, X. Wang, B. Adhi, C. Cortes, J. Anderson, and K. Sano, “Exploration of Trade-offs Between General-Purpose and Specialized Processing Elements in HPC-Oriented CGRA,” in *Proc. IEEE Int. Parallel Distrib. Process. Symp. (IPDPS)*. IEEE, 2024.
- [10] , “ADRES: An architecture with tightly coupled VLIW processor and coarse-grained reconfigurable matrix,” in , 2004, kU Leuven.
- [11] —, “HyCUBE: A CGRA with reconfigurable single-cycle multi-hop interconnect,” in , 2017, nTU Singapore.
- [12] —, “Verification of coarse-grained reconfigurable arrays through random test programs,” in .
- [13] —, “DVHetero: A framework for designing and validating heterogeneous SoC with RISC-V processor and CGRA,” in .
- [14] R. Juneja, P. Dangi, T. K. Bandara, Z. Li, D. Wijerathne, L.-S. Peh, and T. Mitra, “Building an open cgra ecosystem for agile innovation,” in *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2025, pp. 1–9. [Online]. Available: <https://arxiv.org/abs/2508.19090>
- [15] L.-N. Pouchet, “PolyBench/C: The polyhedral benchmark suite,” <https://www.cs.colostate.edu/~pouchet/software/polybench/>, 2012, accessed: 2026-06-18.
- [16] B. Reagen, R. Adolf, Y. S. Shao, G.-Y. Wei, and D. Brooks, “MachSuite: Benchmarks for accelerator design and customized architectures,” <https://github.com/breagen/MachSuite>, 2014, gitHub repository. Accessed: 2026-06-18.